Post-launch connections: analytics, search, behaviour tracking and the deploy that carries them

How a freshly launched site gets its GA4, Search Console and Hotjar wired up, published, verified and handed over. If you remember one thing: **an env change is not live until the next build, and nothing is verified until you have seen the marker in the live HTML**. Ridge & Falls values are left in as a worked example — replace them per site.

SOP ID	HGM-SOP-WEB-003	EFFECTIVE	2026-09-11
VERSION	0.2	NEXT REVIEW	2027-03-11
OWNER	Web & Content Specialist (Kyle Zinger)	TRIGGER	Once per site, in launch week
APPROVED BY	Not yet approved — working draft for Brandon (approver: Operations Manager, Gillian)	DURATION	Some hours, launch week; two working days on

01 PURPOSE AND SCOPE

Purpose

Gets a newly launched site reporting to Google Analytics 4, verified in Google Search Console, recording in Hotjar, and publishes each of those changes through the one deploy path that actually works — then records who owns what at handoff.

In scope

Netlify-hosted Next.js sites deployed through the Netlify connector. The Ridge & Falls launch (2026-09-10 to 09-11) is the worked example throughout; every step here was performed on it.

Out of scope

The launch itself (DNS cutover, go-live), the client-facing analytics walkthrough in `docs/ANALYTICS-SETUP.md`, and paid media tags beyond the Meta Pixel block already in the codebase.

ALWAYS TRUE

Five standing rules

True on every post-launch session.

1

Prove the repo is the live site before you edit

Check DNS, the deploy commit and the HTML, then confirm `.env.local` is filled.

2

Commit, push, then deploy from a clean clone via the connector

A bare `git push` may never publish. Deploy a shallow clone (`git clone --depth 1`, the committed files only) with the connector's `deploy-site`.

3

An env change is not live until the next build

Redeploy after every change to `GA_MEASUREMENT_ID`, `HOTJAR_SITE_ID` or `GOOGLE_SITE_VERIFICATION`, then check the live site.

4

Read the breadcrumb before you create anything in Google Analytics

A property lands in whichever account Admin shows. Move it; recreating it changes its measurement ID (the `G-…` code in `GA_MEASUREMENT_ID`).

5

A new asset is a new filename, always

Assets are cached for a year, so a changed image ships under a new name.

02 PREREQUISITES

1 **A clone of the site repo**

For Ridge & Falls: `github.com/hgmwebteam/ridge-and-falls`, branch `main`. Set-up steps are in HGM-SOP-WEB-002, Phase 1.

2 **The Netlify connector**

Connected in your Claude environment with access to the Netlify team that owns the site. It provides `get-deploy-for-site`, `deploy-site` and `manage-env-vars`.

3 **A filled .env.local**

Restored from the site's 1Password entry (HGM-SOP-WEB-001). Task 1.3 checks it is actually filled, not a copy of the template.

4 Google logins

`kyle@` and `hgmwebteam`. The agency GA account is **Hidden Gem Media**, ID `407691755`. One of these logins must end up holding both Search Console ownership and GA Editor access (task 5.7).

5 The agency Hotjar login

`hgmwebteam`. It holds one organisation per client (Hotjar's container for that client; plan limits and seats are per organisation); sites are added inside the client's organisation, never at the top level.

6 Terminal tools

`git`, `dig`, `curl`, `npx`, and a real browser with no ad blocker for the Hotjar check.

Recommended default

New to this site? Use the Claude prompt for Phases 1 and 2 — it checks the things the raw commands do not, and every Phase 2 deploy goes through the connector anyway. **Comfortable in a terminal?** Type the checks yourself; use Claude for 2.3 and 2.5, which need the connector regardless.

03 PROCEDURE

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1

Prove you are on the live site

Tasks 1.1–1.4 · start of every post-launch session

PHASE 2

Deploy so it actually publishes

Tasks 2.1–2.5 · every code or env change

PHASE 3

Analytics hooks in the code

Tasks 3.1–3.2 · once per engine

PHASE 4

Google Analytics 4

Tasks 4.1–4.10 · property, tag, settings, verify

PHASE 5

Google Search Console

Tasks 5.1–5.7 · verify, sitemap, owner, GA link

PHASE 6

Hotjar

Tasks 6.1–6.6 · client organisation, install, privacy

PHASE 7

Structured data check

Tasks 7.1–7.2 · list page and detail page

PHASE 8

Ownership map at handoff

Tasks 8.1–8.2 · who holds what, what is pending

Phase 1 — Prove you are on the live site

Start of every post-launch session. Ten minutes on Ridge & Falls; it would have prevented two false alarms.

1.1 Confirm the repo is the live site

Check DNS, compare live HTML with the host's HTML, then compare the host's published commit (Netlify connector **get-deploy-for-site** → `commit_ref`) with `origin/main`.

TERMINAL / CLI — REPLACE <DOMAIN> AND <SITE-NAME>

```
dig +short NS <domain>; dig +short A <domain>
curl -s https://<domain>/ > /tmp/live.html
curl -s https://<site-name>.netlify.app/ > /tmp/host.html
diff -q /tmp/live.html /tmp/host.html
git fetch origin && git rev-parse origin/main   # compare with commit_ref
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
I'm starting a post-launch session on this site. Before I touch
anything, prove the repo is the live site:

1. Run dig for NS and A on <domain> and tell me where DNS points.
2. Fetch https://<domain>/ and the Netlify URL for this site and
   compare the HTML. Report any difference.
3. Ask the Netlify connector for get-deploy-for-site and compare
   commit_ref with origin/main after a fetch.

Make no changes. Tell me clearly if any of the three do not match.
```

Expected result

DNS resolves to the Netlify site, `diff` is silent, and `commit_ref` equals `origin/main`. Ridge & Falls: Netlify site `ridge-and-falls`, commit matched.

1.2 Fetch before you trust anything local

A second developer pushes straight to `main` and deploys by API. On Ridge & Falls the local clone was 36 commits behind. After pulling, re-read the project docs (`CLAUDE.md`, the team manual) from the new HEAD before touching code.

TERMINAL / CLI — TYPE IT YOURSELF

```
git fetch && git status -sb
# if behind:
git pull --ff-only
```

POINT & CLICK — THEN RE-OPEN CLAUDE.MD

```
VS Code: Source Control (left sidebar) → the ... menu → Pull
If it offers Sync Changes, click that instead.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch origin and tell me how far behind main is. If it is
behind and a fast-forward is possible, pull. If it is not
a fast-forward, STOP and show me why. Then summarise
what changed in CLAUDE.md and the team manual since my
local HEAD.
```

Expected result

`git status -sb` shows `main...origin/main` with no `[behind N]`.

1.3 Check .env.local is actually filled

Print each key with the **length** of its value, never the value. On Ridge & Falls the file had 30 keys and 28 were blank — a copy of the template. If keys are blank, restore the file from the host's environment variables (or the 1Password entry, HGM-SOP-WEB-001) and restart the dev server.

TERMINAL / CLI — LENGTHS ONLY

```
awk -F= '/^[A-Za-z_][A-Za-z0-9_]*=/ { print $1, length($2) }' .env.local
```

POINT & CLICK — LOOK, DON'T COPY

```
VS Code: open .env.local from the Explorer.
Scan the right-hand side of each line — any KEY= with nothing
after the = is blank.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Check .env.local: list every key with the LENGTH of its value.
Never print a value. Tell me which keys are blank. If keys
are blank, tell me which host environment variables would
fill them, but do not fetch or write any secret yourself.
```

What a blank env looks like

Empty listings, missing photos, "No properties match". No Supabase URL means the cache is skipped; no Guesty credentials means the live read fails once the cached token expires. It looks like a broken site. It is a blank file.

Never print or paste values

Check lengths, not contents. Do not paste an env value into Google Chat, an Asana comment or a Claude prompt.

Expected result

Every key that the site needs shows a non-zero length, and the dev server renders listings with photos.

1.4 Tell the team the Guesty quota before anyone tests locally

Guesty mints about five OAuth tokens per 24 hours per client ID, and production shares that allowance. Browsing home, listings and property pages locally reads the Supabase cache and spends nothing. Quotes and checkout mint a token. Say this before anyone "tests locally".

Expected result

Nobody runs a local checkout without knowing it costs a production token.

Phase 2 — Deploy so it actually publishes

Used by every later phase. Tasks 2.1–2.4 publish a code change. Task 2.5 is how an env var is set — and it always ends by running 2.2–2.4 again.

2.1 Commit and push to main

Always commit first, so the deploy is of a recorded tree.

TERMINAL / CLI — TYPE IT YOURSELF

```
git push origin main
```

POINT & CLICK — PUSHES TO MAIN

```
VS Code: Source Control → type a message → Commit → Sync Changes
(or ... → Push)
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Commit my staged changes with the message I give you, then
push to origin main. Confirm the push landed by showing me
the new HEAD on origin/main. Do not deploy yet.
```

> **The push alone may never publish**
>
> A push to `main` from a committer Netlify has not verified sits in review, invisible from our tooling. On Ridge & Falls we waited six minutes and nothing happened. Do not wait for it — go to 2.2.

2.2 Make a shallow clone of the committed tree

Never deploy from the repo root. The root upload includes `.next`, `RESOURCES/` and `.env.local` — about 700 MB — and Netlify returns 500s.

TERMINAL / CLI — TYPE IT YOURSELF

```
git clone -q --depth 1 "file://$PWD" /tmp/rf-deploy && cd /tmp/rf-deploy
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Make a shallow clone of the committed tree for deployment:
git clone --depth 1 from file://$PWD into /tmp/rf-deploy.
Confirm .next, RESOURCES/ and .env.local are NOT in the clone.
```

Expected result

You are in `/tmp/rf-deploy` with only tracked files present.

2.3 Request a proxy URL from the connector, then deploy

Ask the Netlify connector for **deploy-site**; it returns a one-time proxy URL. Run the deploy from the shallow clone with that URL. Each proxy URL is single-use — request a fresh one per deploy.

TERMINAL / CLI — RUN INSIDE /TMP/RF-DEPLOY

```
npx -y @netlify/mcp@latest --site-id <site-id> --proxy-path "<proxy-url>"
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Deploy this site from /tmp/rf-deploy using the Netlify connector:

1. Call deploy-site for site <site-id> and get a fresh proxy URL.
2. Run npx -y @netlify/mcp@latest --site-id <site-id>
   --proxy-path "<that url>" from inside /tmp/rf-deploy.
3. Tell me when the connector reports ready.

Never deploy from the repo root. If the clone is missing, stop
and tell me — do not create it from the working tree.
```

Expected result

The connector reports the deploy as ready. Do not stop here — 2.4 is the real check.

2.4 Verify against production for a marker only the new build has

A new filename, a new meta tag, the GA ID. "Deploy is ready" is not verification; cached pages can lag a minute. Repeat the curl until the marker appears.

TERMINAL / CLI — REPLACE <MARKER>

```
curl -s https://<domain>/ | grep -c "<marker>"
```

POINT & CLICK — THEN CMD + F

```
Browser: open https://<domain>/ → View Page Source
(Cmd + Option + U in Chrome) → Cmd + F for the marker.
Hard refresh with Cmd + Shift + R if it is missing.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch https://<domain>/ and tell me whether the string
"<marker>" is in the live HTML. If not, retry every 30 seconds
for up to 3 minutes and report when it appears.
```

Why we poll HTML and not the API

The `NETLIFY_AUTH_TOKEN` in the local env file returned 401 from Netlify's API, so deploys could not be polled by API. Polling the live HTML for the marker works regardless.

Expected result

`grep -c` returns 1 or more.

2.5 Set or change an environment variable, then redeploy

This needs the Netlify connector, so it is a Claude prompt route only. Ask for **manage-env-vars**, action **upsert**, context **production**, then run tasks 2.2 to 2.4 again — the head is rendered at build time, so the change is invisible until the next build.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Set an environment variable on the production site with the
Netlify connector: manage-env-vars, action upsert, context
production, key <NAME>, value <VALUE>. Then redeploy from a
fresh shallow clone (tasks 2.2 to 2.3) and confirm the new
value is in the live HTML. Do not put this variable in
.env.local.
```

Production context only

The three analytics variables are set in Netlify's Production context and never in `.env.local`, so dev and preview traffic does not report into the client's analytics.

Expected result

After the redeploy, the marker for the new value (the `G-…` ID, the Hotjar ID, the verification token) is in the live HTML.

Phase 3 — Analytics hooks in the code

Once per engine. If the site already carries the three env-gated blocks (pieces of `app/layout.tsx` that render only when their Netlify variable is set), confirm them and move on.

3.1 Confirm the three env-gated blocks exist in app/layout.tsx

They are modelled on the existing Meta Pixel block. Each reads a server-side env var (no `NEXT_PUBLIC_` prefix), shape-checks the value before interpolating it, and renders nothing while the var is unset.

ENV VAR (NETLIFY, PRODUCTION ONLY)	VALUE SHAPE	RENDERS
`GA_MEASUREMENT_ID`	`G-…`	gtag.js plus `gtag('config')`, strategy `lazyOnload`
`HOTJAR_SITE_ID`	digits	Hotjar snippet (hjsv 6), strategy `lazyOnload`
`GOOGLE_SITE_VERIFICATION`	token	`<meta name="google-site-verification">` via Next `metadata.verification.google`

Expected result

All three blocks are present and gated. If any is missing, build it on the Meta Pixel pattern before continuing.

3.2 Confirm the names are documented and the purchase event is absent

The three variable names appear in `.env.example` and in the docs' env list. There is **no** GA4 purchase event, by design.

Why there is no browser purchase event

The site reports bookings server-side to Meta with a dedupe key. A browser purchase event in GA4 would double count. Do not add one.

Expected result

`grep` for each name in `.env.example` finds it; `grep` for `purchase` in `app/layout.tsx` finds nothing.

Phase 4 — Google Analytics 4

Where the property lives is the part that cost the most time on Ridge & Falls. Tasks 4.1 and 4.2 are about that; 4.3 onward are the settings and the check.

4.1 Read the breadcrumb, then create the property under the agency account

In GA, open **Admin**. Read the account name in the breadcrumb at the top of the Admin page. Switch to **Hidden Gem Media** (`407691755`) if it is not showing. Only then choose **Create → Property**. Name it after the client's site and add a web data stream for the apex domain.

The property lands in whichever account Admin is showing

On Ridge & Falls it landed under another client's account twice. The breadcrumb is the only thing that tells you where you are.

The agency pattern

Create under Hidden Gem Media, make the client Administrator on the property (4.2), and let the client move it into their own account — they administer the destination, so the move works for them. Our property-level access travels with it.

Expected result

A new property with a `G-…` measurement ID under Hidden Gem Media. Ridge & Falls: property "Ridge & Falls", stream ridgeandfalls.com, `G-PYNW47J2EK`.

4.2 Add the client as Administrator on the property and ask them to move it

Admin → Property access management → Add users, role **Administrator**. Then ask the account manager to have the client move the property: **Admin → Property details → Move property**, choosing their account by its **9-digit account ID**.

Why the move failed from our side

Moving needs Administrator on the destination account, which we do not hold. There were also two accounts named for the client — pick by ID, not name. If the **Move property** option does not appear for the client, ask for temporary Administrator on their account instead.

Never delete and recreate

Recreating a property changes the measurement ID, which means an env change and a redeploy. Moving keeps the ID and the data.

Expected result

The client appears as Administrator. Ridge & Falls: fred@enjoyuniquestays.com is Administrator; the move into Enjoy Unique Stays (`364107411`) is pending with the account manager.

4.3 Set GA_MEASUREMENT_ID and redeploy

Run task 2.5 with `GA_MEASUREMENT_ID` = the `G-…` ID from 4.1. Then view source on production.

TERMINAL / CLI — TYPE IT YOURSELF

```
curl -s https://<domain>/ | grep -o "gtag/js?id=G-[A-Z0-9]*"
curl -s https://<domain>/ | grep -o "gtag('config', 'G-[A-Z0-9]*')"
```

POINT & CLICK — VIEW SOURCE

```
Browser: View Page Source on https://<domain>/ → Cmd + F
"gtag/js?id=" — the G- ID next to it must match the one you set.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch https://<domain>/ and tell me the G- measurement ID in
both the gtag/js script URL and the gtag('config') call.
Both must equal <G-ID>.
```

Expected result

Both greps print the ID you set.

4.4 Turn on page views for browser history events

Admin → Data streams → your stream → **Enhanced measurement** → gear icon → **Page views → Show advanced settings → Page changes based on browser history events**: **ON**. The site fires one page view per full load and relies on this setting for client-side navigations.

Expected result

The toggle shows on and the setting is saved.

4.5 Set data retention to 14 months

Admin → Data collection and modification → Data retention → Event data retention: **14 months**. The default is 2.

Expected result

Retention reads 14 months.

4.6 Set the property time zone and currency

Admin → Property details. Time zone = the site's, from `BRAND.timezone` in the codebase (Eastern for Ridge & Falls). Currency **USD**.

Expected result

Property details show the site's time zone and USD.

4.7 Define internal traffic and set the filter to Active

Admin → Data streams → stream **→ Configure tag settings → Define internal traffic**: add the office and home IPs. Then **Admin → Data filters → Internal Traffic →** state **Active**.

> ⚠ **Filters ship in Testing**
>
> A filter in Testing does nothing to reports. It must read Active.

> 👁 **Expected result**
>
> The Internal Traffic filter shows Active.

4.8 Leave Google Signals off unless the client wants demographics

Admin → Data collection and modification → Data collection → Google signals data collection. Leave off by default; turn on only on the client's request.

> 👁 **Expected result**
>
> Signals off, or on with the client's request recorded in `CLAUDE.md`.

4.9 Optional: create an approximate booking count event

Admin → Events → Create event. Name `booking_confirmed`. Conditions: `event_name` equals `page_view` AND `page_location` contains `/confirmation`. Then mark it as a key event. This is approximate and carries no value.

> 👁 **Expected result**
>
> `booking_confirmed` appears under Events and is marked as a key event.

4.10 Verify in Realtime with a real visit

Open the live site in a browser, then **Reports → Realtime**. You should appear within a minute.

Two things that look like failure and are not

The "Data collection isn't active" banner can take 48 hours to clear — ignore it if Realtime works. Google's **Test your website** button, and GA-based Search Console verification, may not see the tag because it loads after `window.load`.

Expected result

One active user in Realtime, on the page you opened.

Phase 5 — Google Search Console

A Domain property (every protocol and subdomain, verified only by a DNS TXT record) if you can reach DNS; a URL-prefix property (one exact address) with an HTML tag if you cannot.

5.1 Decide which property type you can verify

Find where the domain's DNS is hosted (`dig +short NS <domain>` from 1.1). **If the agency holds that DNS login** → add a **Domain** property in Search Console and verify with the DNS TXT record it gives you, then go to 5.4. **If not** → go to 5.2.

Ridge & Falls

Registrar is Squarespace Domains; DNS is at **Cloudflare**; nobody at the agency holds that login. The Domain property is parked unverified and one TXT record will verify it later.

5.2 Add a URL-prefix property and choose the HTML tag method

Add property → URL prefix, enter `https://<domain>` (the apex; www and http redirect to it, so coverage is equivalent). Under verification methods choose **HTML tag** and copy the `content` token only — not the whole tag.

The Domain-property dialog reopens every visit

Until you click **try a URL prefix property instead**. That is not a bug.

Expected result

A token string on your clipboard. Ridge & Falls property: `https://ridgeandfalls.com`.

5.3 Set GOOGLE_SITE_VERIFICATION, redeploy, then click Verify

Run task 2.5 with `GOOGLE_SITE_VERIFICATION` = the token. Confirm the meta tag is in the live HTML, then return to Search Console and click **Verify**.

TERMINAL / CLI — TYPE IT YOURSELF

```
curl -s https://<domain>/ | grep -o 'name="google-site-verification"[^>]*'
```

POINT & CLICK — VIEW SOURCE

```
Browser: View Page Source → Cmd + F "google-site-verification"
→ the content= value must match the token you copied.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch https://<domain>/ and show me the content attribute of
the google-site-verification meta tag. Tell me whether it
matches <TOKEN>.
```

The token is per Google account

If a second login needs verified status, the site can carry a second token — Next accepts an array for `metadata.verification.google`.

Expected result

Search Console shows the property as verified.

5.4 Submit the sitemap

Sitemaps → Add a new sitemap: `sitemap.xml`. Noindex pages are excluded from it on purpose.

Expected result

Status Success. Ridge & Falls: 13 URLs.

5.5 Request indexing for the home page and the listings page only

URL Inspection → enter `https://<domain>/` → **Request indexing**. Repeat for `/listings`. Stop there: the daily quota is small and the sitemap covers the rest.

Expected result

Both URLs show "Indexing requested".

5.6 Add the client as Owner

Settings → Users and permissions → Add user, permission **Owner**.

An added Owner depends on our token staying valid

The client becomes independent only once they verify with their own TXT record or tag. Do not remove the agency's record until they have.

Expected result

The client's login is listed as Owner.

5.7 Link GA4 to Search Console from one login that holds both

The link must be created by a Google account that has verified ownership in Search Console **and** Editor or above on the GA property. If they are split across two logins, share Search Console ownership to the GA login first. Then **GA Admin → Product links → Search Console links → Link**.

Ridge & Falls

kyle@ and hgmwebteam were both in play. Sharing GSC ownership to the GA login resolved it.

Expected result

The Search Console link is listed under Product links.

Phase 6 — Hotjar

Agency account, client organisation, keystrokes off and exclusions set before the first recording, real browser visit to verify.

6.1 Open the client's organisation in the hgmwebteam Hotjar account

Log in as `hgmwebteam`. Switch to the client's organisation. If none exists, choose **Add as a New Client Organization** and name it after the **client**, not the site — plan limits and team seats are per organisation.

Expected result

You are inside an organisation named for the client.

6.2 Survey the client's sibling sites for an existing hjid

Read each sibling site's HTML and any Tag Manager container for `hjid`, so you do not create a second site for one that already runs Hotjar.

TERMINAL / CLI — TYPE IT YOURSELF

```
curl -s https://<sibling-domain>/ | grep -o "hjid:[0-9]*"
```

POINT & CLICK — VIEW SOURCE

```
Browser: View Page Source on each sibling site → Cmd + F "hjid".
A hit shows the existing Hotjar site ID; no hit means none.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
For each of these domains, fetch the home page and report
whether it loads Hotjar and, if so, the hjid value:
<domain-1>, <domain-2>, <domain-3>.
Also check any Google Tag Manager container you can see.
```

Expected result

A list of which sibling sites carry Hotjar and their IDs. Ridge & Falls survey: only Paradise Pointe (site `6626800`) ran Hotjar; the rest none.

6.3 Add the site, then set HOTJAR_SITE_ID and redeploy

Inside the client organisation, add the site for the apex domain and copy its numeric Site ID. Run task 2.5 with `HOTJAR_SITE_ID` = that ID.

Expected result

The Hotjar snippet with your ID is in the live HTML. Ridge & Falls: `6776829`.

6.4 Verify in a real browser, then confirm the loader

Open the live site in a browser with **no ad blocker** (Brave and uBlock hide it) and browse two pages. Hotjar marks a site installed only after its script runs in a real browser — `curl` does not count. Separately, confirm the loader URL is valid.

TERMINAL / CLI — TYPE IT YOURSELF

```
curl -s -o /dev/null -w "%{http_code}\n" "https://static.hotjar.com/c/hotjar-<id>.js?sv=6"
curl -s "https://static.hotjar.com/c/hotjar-<id>.js?sv=6" | grep -o "site_id[^,]*"
```

POINT & CLICK — REAL BROWSER

```
Browser (no ad blocker): open https://<domain>/, click through
two pages. Then open
https://static.hotjar.com/c/hotjar-<id>.js?sv=6 in a tab —
it should load and mention site_id: <id>.
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch https://static.hotjar.com/c/hotjar-<id>.js?sv=6 and
report the HTTP status and the site_id in the response.
Remind me that Hotjar only marks the site installed after a
real browser visit, and that curl does not count.
```

Expected result

Hotjar shows the site as installed; the loader returns 200 with the right `site_id`.

6.5 Fix the privacy settings before the first recording

In the site's settings: turn **keystroke capture off** (it was on by default on Ridge & Falls). Add URL exclusions for paths containing `/admin`, `/manual`, `/checkout` and `/confirmation`. Add the office IPs to **IP blocking**.

The confirmation page prints guest name and email as text

Without the exclusion, a session recording captures personal data. Set the exclusions before any real traffic is recorded.

Expected result

Keystrokes off, four exclusions listed, office IPs blocked.

6.6 Invite the client and set expectations on heatmaps

Team → Invite the client's login. Tell them: the Basic plan samples about 10% of sessions; heatmaps are on-demand views, not objects to create — enter the **full URL**, use **starts with** for `/listings` (filtered views) and **contains** for `/property/` (all cabins pooled); data appears after a few dozen visits.

Expected result

The client shows as invited in Team.

Phase 7 — Structured data check after launch

Test the list page, not just a detail page. That is where the duplicate showed up.

7.1 Run the Rich Results Test on the listing page and a detail page

Open Google's Rich Results Test and test `/listings` (or the site's collection page) first, then one property detail page.

What went wrong on Ridge & Falls

The property page emitted a full `VacationRental` node while the listings `ItemList` emitted a thin copy with the same `@id`. The test merged them and scored every cabin "2 critical".

Expected result

Zero critical issues on both pages. If the list page fails → 7.2.

7.2 Fix with one builder function per entity

Use a single builder for the entity — `vacationRentalJsonLd` in `lib/seo.ts` — from every page that mentions it. Fill the required property set from the schema spec page itself (fetched, not remembered) and the recommended set where the data is real. Leave out anything that would be invented, such as review dates or bed types. Then deploy (Phase 2) and re-run 7.1.

Say plainly when a rich result is partner-gated

Google's vacation-rental rich result is for Hotel Center partners. The value here is entity completeness for search and answer engines, not a guaranteed rich result.

Expected result

Both pages pass with zero critical issues.

Phase 8 — Ownership map at handoff

Fill this per site. The Ridge & Falls map as of 2026-09-11 is the worked example.

8.1 Fill the ownership map

One row per thing the client will eventually need to control. Record where it lives, who has access today, and what is pending.

THING	WHERE IT LIVES	WHO HAS ACCESS	PENDING
Code	`github.com/hgmwebteam/ridge-and-falls`, `main`	HiddenGem	—
Hosting	Netlify site `ridge-and-falls` (`89eb273f…`), primary ridgeandfalls.com	Netlify team; connector deploys	—
Domain	Registrar Squarespace Domains; DNS at Cloudflare	**Unknown login**	Find the Cloudflare owner
GA4	Property "Ridge & Falls", `G-PYNW47J2EK`, in Hidden Gem Media (`407691755`)	kyle@ (Admin), fred@ (Admin)	Move into Enjoy Unique Stays `364107411`
Search Console	URL-prefix `https://ridgeandfalls.com` (HTML tag); Domain property parked	kyle@, hgmwebteam (Owners)	Client verification; DNS TXT for the Domain property
Hotjar	Site `6776829` inside the client organisation, hgmwebteam account	hgmwebteam	Client invite; privacy settings confirmed
Env vars	Netlify (Production): `GA_MEASUREMENT_ID`, `HOTJAR_SITE_ID`, `GOOGLE_SITE_VERIFICATION`	—	—
Privacy copy	`lib/privacy-content.ts`	—	Client wording for analytics and session recording

Expected result

Every row filled; every Pending item has a named owner in Asana.

8.2 Record the decisions, then publish the final state

Ask the account manager to request the privacy wording from the client. Record the day's decisions in `CLAUDE.md` and the team manual's Updates. Commit, push, deploy from a shallow clone (Phase 2) and verify the live markers one last time. Post the ownership map in the **client websites** space in Google Chat: post-launch hand-offs go there.

Expected result

`CLAUDE.md` carries the decisions; the live HTML carries the GA ID, the Hotjar ID and the verification tag.

A client change request arrives during launch week

The pattern, in order. Every example shipped on Ridge & Falls followed it.

1. **Locate the code from the screenshot or chat request**
   If the screenshot shows something "missing" on a dev box, check the data feed before the code — see task 1.3.

2. **Edit, then run the checks**
   `tsc`, `vitest` and `verify:brand`.

3. **Render through the real stylesheet for a one-look check**
   Use the Playwright harness so you are looking at the actual output, not a guess.

4. **Record the decision**
   In `CLAUDE.md` and the team manual's Updates.

5. **Commit, deploy, verify live**
   Phase 2, including the marker check in 2.4.

A changed asset is a new filename

Static assets are cached immutable for a year. Ship `wordmark-light-compact.svg`, `collection-*.webp`, `local-favorites-mclemore-resort-v3.webp` — never overwrite in place.

Keep the client's swatches even when contrast fails

Document the contrast ratio at the declaration and tell the client the passing alternative. Do not silently change their colour.

Blur is often the sizes hint, not the file

The Local Favorites portrait crop was blurry because of the `srcset sizes` hint, not the images. Check the hint before regenerating assets.

If you are stuck

Ask Claude first, with the task number and what you see. Then Kyle Zinger (Web & Content Specialist) in the **Web Team** Google Chat group. Anything that needs the client — the GA move, the Cloudflare login, the privacy wording — goes through the client's account manager, never the client directly.

05 FINAL CHECKLIST BEFORE HANDOFF

- [ ] Repo = live site confirmed (DNS, deploy commit, HTML compare)
- [ ] Local clone fast-forwarded to origin; local env filled (lengths checked, not values)
- [ ] Analytics hooks present in `app/layout.tsx`; env names documented
- [ ] GA4 property created in the right account (breadcrumb checked); time zone and currency set
- [ ] `GA_MEASUREMENT_ID` set in Production → deployed → tag in source → Realtime shows a visit
- [ ] Enhanced measurement history events on; retention 14 months; internal traffic filter Active
- [ ] Client is Administrator on the GA property; move requested if needed
- [ ] Search Console property verified (DNS if possible, else HTML tag via env var and deploy)
- [ ] Sitemap submitted; home and listings requested for indexing
- [ ] Client added as Search Console Owner; GA to GSC link created from a login holding both
- [ ] Hotjar site inside the client's organisation; `HOTJAR_SITE_ID` set → deployed → verified in a real browser
- [ ] Hotjar keystrokes off, private pages excluded, office IPs blocked, client invited
- [ ] Rich Results Test on list page and detail page: zero critical
- [ ] Ownership map filled; pending items assigned; privacy copy requested from the client
- [ ] `CLAUDE.md` and manual updated; committed; pushed; deployed; live markers verified

06 RELATED DOCUMENTS

- `docs/ANALYTICS-SETUP.md` in the site repo — the client-facing walkthrough for GA4, Search Console and Hotjar
- `CLAUDE.md` decisions 28–29 — the engineering record of the same two days
- HGM-SOP-WEB-001 — env files in 1Password
- HGM-SOP-WEB-002 — take a website change request from Asana to live (branch, preview, merge)
- `SOP-POST-LAUNCH-CONNECTIONS.md` — the source notes this draft was converted from

07 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE SUMMARY
2026-09-11	0.1	Kyle Zinger	Working draft converted from the Ridge & Falls launch-week notes, for Brandon. Not yet approved.
2026-10-15	0.2	Kyle Zinger	Loom review layout (house style v1.9). Roles, definitions and troubleshooting removed: terms are explained where first used, and each fix sits in the task it belongs to. Standing rules shortened; prerequisites collapse; an If you are stuck note; a copy-to-Claude button on the final checklist; links to other SOPs open in the portal.

REMEMBER

"Prove you are looking at the live site before you change it. Then remember that nothing you set is live until the next build, and nothing is verified until you have seen it in the live HTML."

Questions?
Kyle — Web Team Google Chat group